Opgave 1

Ydre løkke (linie 9) O(N), da i tælles op med en for hvert gennemløb

Midt løkke (linie 10): log(N) da j ganges med to for hvert gennemløb (linie 14)

Indre løkke: O(N*sqrt(N)) da k tælles op til N*sqrt(N)

Samlet: O(N*log(N)*N*sqrt(N)) = O(N^2*log(N)*sqrt(N))

Anden del (linie 18-20): O(N^2) da i tælles op til N^2. De to dele er i sekvens så den samlede tidskompleksitet er den største af de to funktioner: O(N^2*sqrt(N)*log(N)) da både log(N) og sqrt(N) er positive og voksende

Opgave 2

```
static int oddSquare(int N) {
  assert(N>=1); //ikke nødvendig
  if (N==1) return 1;
  else {
    if (N%2 == 0) N-=1; //her spares halvdelen af recursive kald
    return N*N+oddSquare(N-2);
  }
}
```

Opgave 3

1. Ja det er et binært søgetræ, da alle knuder overholder kravet (knuder i venstre undertræ er mindre end knuden, knuder i højre undertræ er større)
2. inorder: 1, 2, 3, 9, 11, 13, 17, 25, 57, 90
   preorder: 11, 2, 1, 9, 3, 13, 57, 25, 17, 90
3. Ved knuden "13" er højdeforskellen 3. roter så højdeforskellen er max 1:

```
   17
  / \
13   57
     / \
   25   90
```

4. nej, da hver operation kun kan ændre højdenforskellen med én

Opgave 4

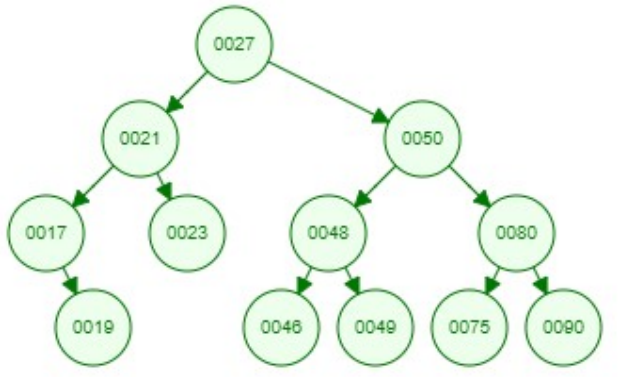
```
class MaxHeap {
private:
  std::vector<int> v;
public:
  MaxHeap() {}
  bool isEmpty() {return v.empty();}
  int size() {return v.size();}
  void insert(const int x) //add an element to the heap
  { v.push_back(x);}
  const int findMax() const // find the maximum element of the heap
  {
    std::vector<int>::const_iterator it = v.begin();
    int max= *it;
    for ( ; it != v.end(); ++it)
      if (*it>max) max = *it;
    return max;
  }
  int deleteMax() //delete and return the maximum element of the heap
  {
    std::vector<int>::const_iterator it = v.begin();
    std::vector<int>::const_iterator max= it;
    for ( ; it != v.end(); ++it)
      if (*it>*max) max = it;
    int m = *max;
    v.erase(max);
    return m;
  }

};
```

Opgave 5

1. knude i's børn befinder sig i index 2i og 2i+1. Herved kan vi traversere i O(log(N)) tid. Roden fides i index 1.
2. Nej. 80 står forkert (er angivet som 17's venstre barn dvs mindre end 17). 75 og 90 har ingen parent

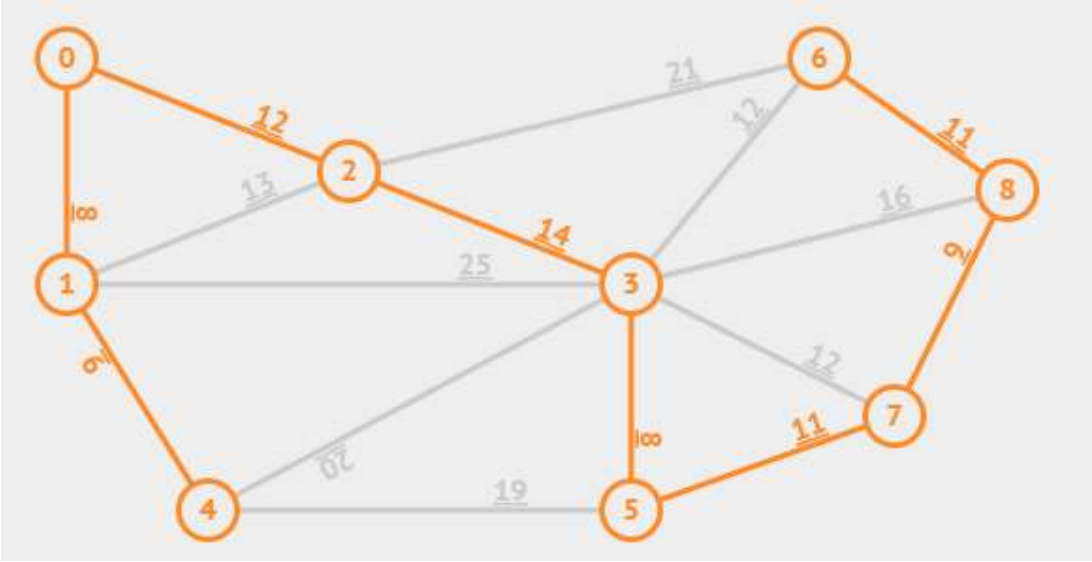
3.

	27	21	50	17	23	48	80	-1	19	-1	-1	46	49	75	90
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15



4. pre-order gennemløb. O(N) hvor N er antallet af knuder

<u>Spørgsmål 6</u>



Rækkefølge: (0,1), (3,5) (eller omvendt), (1,4), (7,8) (eller omvendt), (7,5), (8,6) (eller omvendt), (0,2), (2,3)